

백엔드에서 쌓은 판단을 AI Native 개발 방식으로 구조화합니다

Java와 Kotlin으로 이벤트·배치·검색·캐시 시스템을 설계하고 운영해 왔습니다. 그 과정에서 쌓은 판단을 에이전트·스킬·평가로 구조화해 실제 개발 흐름에서 재사용합니다.

서민재 · Rex Seo

AI-Native
Backend
Engineer ckdekrn88@gmail.com

GitHub Velog

LinkedIn 이력서

경력기술서

01

POINT PLATFORM · EVENT RELIABILITY

발행 실패와 처리 실패, 같은 방식으로 복구할 수 없었습니다.

포인트 차감이나 회수가 늦게 실패하면 정산 확인과 고객 응대도 함께 늦어집니다. 실패 지점과 복구 대상을 이벤트 레코드에 남겨, 로그를 처음부터 추적하지 않고 멈춘 업무부터 다시 실행할 수 있게 했습니다.

담당

이벤트 상태·수동 복구·정체
알림 설계 및 구현

구조

내부 이벤트 9종, 상태 6개,
타입별 복구 전략 8개

저장

PostgreSQL
internal_event_record ·
JSONB payload

관측

IN_PROGRESS 15분 초과
건을 타입별로 집계해 Slack
전송

상황

‘실패’ 하나로 묶자, 원인을 찾는 일부터 다시
시작해야 했습니다.

메시지를 보내지 못한 경우와 소비자가 처리 중 멈춘 경우는
확인할 데이터도, 재시도할 위치도 다릅니다. 상태가 같으면
운영자가 로그와 원장을 일일이 맞춰 봐야 했습니다.

판단

멈춘 지점을 기록하고, 복구 로직은
업무별로 나눴습니다.

생성부터 복구 불가 실패까지 6개 상태를 두었습니다.
포인트 차감·회수·만료 알림처럼 조건이 다른 8개 작업은
저장된 payload로 각 업무 서비스를 다시 호출합니다.

멈춘 위치에서 다시 시작하는 복구 흐름

생성부터 처리, 수동 복구와 정체 감지까지

—— 호출 - - - 비동기 — 복구

01 정상 처리

업무 요청과 처리 상태를 기록한 뒤 Kafka로 전달합니다.

업무 요청

API · Admin · Batch

차감 · 회수 · 사용 · 만료

기록

상태 기록

internal_event_record

InternalEventCreator

CREATED · JSONB payload

전달

KAFKA

내부 이벤트 topic 9개

발행 실패는 SEND_FAILED로 분리

처리

업무 처리

Listener 9개 · EventProcessor

IN_PROGRESS → 완료 · 실패

02 수동 복구

Kafka에 다시 넣지 않고 실패한 업무만 실행합니다.

복구 대상 조회

type · status로 검색

저장된 payload 복원

선택

복구 전략

Retry Strategy 8개

이벤트 타입별 업무 분기

재실행

DOMAIN SERVICE

업무 서비스 직접 호출

차감 · 회수 · 사용 · 알림

03 정체 감지

15분 넘게 처리 중인 이벤트를 Slack으로 알립니다.

QUERYDSL

IN_PROGRESS 15분 초과

추적 대상 이벤트 8종

집계

SLACK

이벤트 타입별 요약

운영 채널로 건수 전송

달라진 운영

전송 실패와 처리 실패를 서로 다른 상태로 기록합니다. 처리 중 15분 넘게 멈춘 이벤트는 Slack으로 알리고, 저장된 payload로 실패한 업무만 다시 실행합니다.

세부 구현과 예외 처리

- 내부 이벤트 9종을 생성, 발행 실패, 처리 중, 완료, 처리 실패, 복구 불가 실패의 6개 상태로 기록했습니다.
- 수동 복구 API가 타입과 상태로 레코드를 찾고 저장된 payload로 8개 전략 중 하나를 실행하도록 구성했습니다.
- Kafka에 다시 넣지 않고 타입별 업무 서비스를 직접 호출해 복구 범위를 실패한 업무로 제한했습니다.
- IN_PROGRESS 상태로 15분 이상 멈춘 추적 대상 8종만 조회해 타입별 건수를 Slack에 전송했습니다.
- 상태 전환 타이밍 오류, 잘못된 상태 검사, 이벤트 발송 실패 로깅 누락을 수정했습니다.
- 배치 재시도와 프록시 기반 재시도 호출 문제를 수정하고 포인트 선점 복구 경로를 보강했습니다.

배치는 멈춘 위치를, 동시 요청은 끝난 뒤의 상태를 확인했습니다.

원장 소멸·만료 알림·아카이브는 작업마다 다시 시작할 위치가 다릅니다. 포인트 거래와 예산 차감은 동시에 실행된 뒤 잔액과 원장 상태가 맞아야 합니다.

담당	처리 단위	보관 대상	동시성
원장 소멸 페이지 전환· 아카이브 배치 구현·만료 알림 이벤트 개선	전체 적재 대신 페이지 단위 반복 처리	point_core 6개 ·point_service 4개 archive 테이블	9개 API · 13개 시나리오 · 3·5·10개 동시 요청

상황

세 Job이 같은 데이터를 만진다고 같은 방식으로 돌릴 수는 없었습니다.

소멸은 잔액과 원장을 바꾸고, 만료 안내는 알림을 쌓아 발송하며, 아카이브는 참조 순서대로 테이블을 옮깁니다. 전체 적재 방식은 데이터가 늘수록 실패 범위도 함께 커졌습니다.

판단

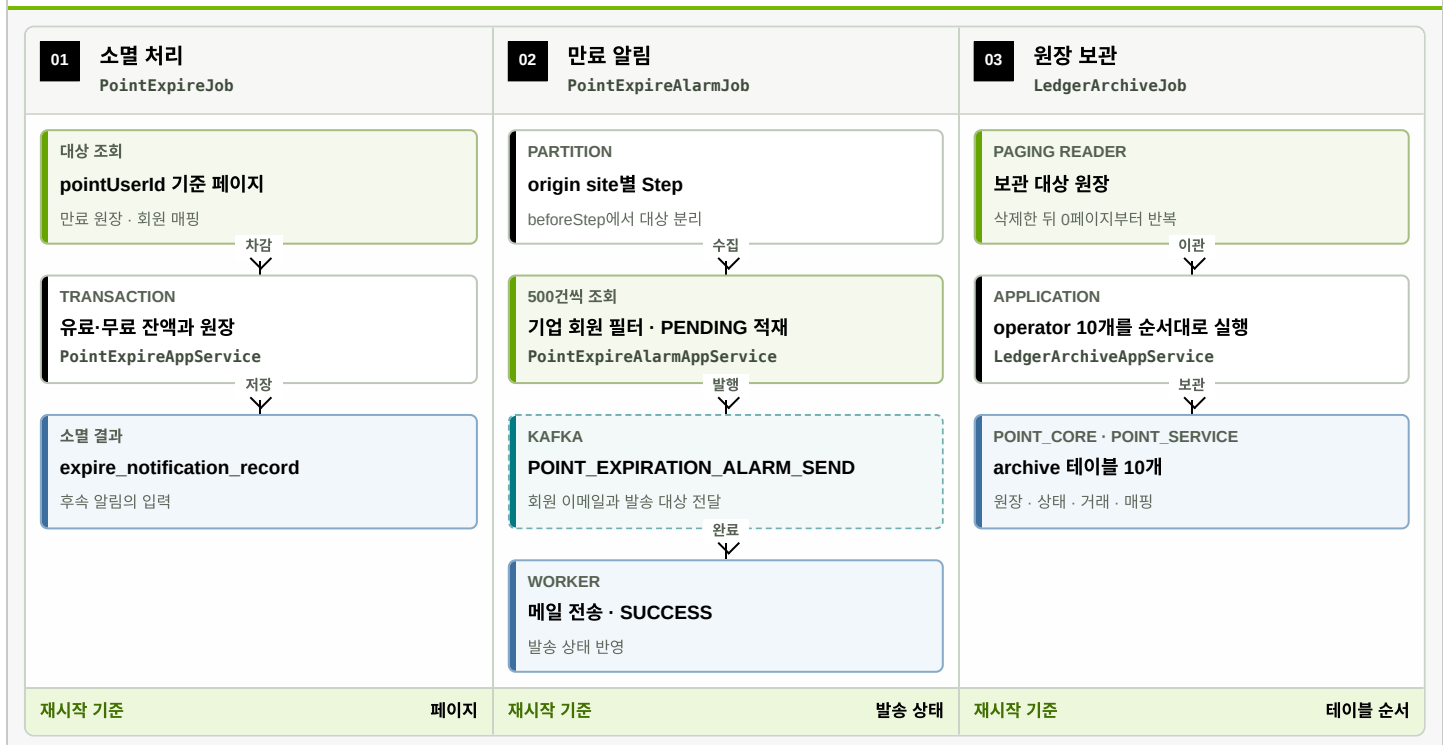
배치에는 재시작 기준을, 동시 요청에는 결과 불변식을 뒀습니다.

소멸·알림·아카이브는 진행 위치를 남겼습니다. 포인트 거래와 예산 차감은 락 획득 여부뿐 아니라 경쟁이 끝난 뒤 잔액·원장 상태·이벤트 수가 맞는지 확인했습니다.

세 배치, 세 가지 재시작 기준

같은 원장을 다뤄도 다시 시작할 기준은 서로 다릅니다.

—— 순차 처리 - - - 후속 이벤트



락 키와 해제 시점에 따라 달라지는 세 가지 경쟁 결과

중복 차단·거래 정합성·어뷰징 분기의 세 경로로 나뉘었습니다.

9개 API

발급 · 거래 · 복구 · 예산 차감

13개 시나리오

같은 락 키 · 다른 거래 참조값 · 유지 시간 경계

3 · 5 · 10개 스레드

연속 재요청은 별도 타임라인으로 검증

01 검증 대상

API마다 락 키와 해제 시점은 달라도 경쟁 후 상태는 설명할 수 있어야 합니다.

1회용 키

중복 발급 차단

포인트 적립

잔액 · 한도

포인트 선점

무료 · 유료 잔액

포인트 사용

원장 · 선점 상태

선점 복구

잔액 · 복구 상태

포인트 환불

원장 · 환불 상태

사용 중단

원장 · 중단 상태

사용 재개

원장 · 재개 상태

클릭 예산

차감 · 어뷰징 이벤트

경쟁 후 검증 기준

성공 건수 · 잔액 · 상태 · 이벤트

02 중복 발급 억제

동일 사용자 · 락 유지 시간

10개 스레드

같은 사용자의 키 발급 요청

같은 식별자로 동시에 진입

차단

기대 결과

동시 요청 10건 · 생성된 키 1개

같은 사용자에게 키가 중복 생성되지 않음

락 유지 시간 경계

경과 후 재요청 허용

03 거래 순서와 정합성

동일 사용자 · 서로 다른 거래 참조값

3개 스레드

API별 서로 다른 거래 정보 3건

적립부터 사용 중단·재개까지 7개 거래 API

순차 처리

기대 결과

3건 모두 완료

성공 여부와 함께 상태를 함께 확인

불변식

잔액 · 원장 · 업무 상태

04 클릭 어뷰징 분기

복합 요청 식별자 · 락 유지 시간

5개 스레드

고유 요청과 중복 요청

복합 요청 식별자로 중복 여부를 구분

판별

기대 결과

고유 요청 5건 · 중복 요청 1건 처리

중복 4건은 대체 처리·어뷰징 이벤트로 전환

후속 처리

차감 · 대체 처리 · 이벤트

잔액

무료·유료 잔액과 적립 한도 누적액

원장·업무 상태

사용 · 복구 · 환불 · 중단 · 재개

후속 이벤트

차감 · 대체 처리 · 어뷰징 이벤트 수

CI 느린 테스트의 실행 경계

DisableSlowTest가 붙은 테스트는 기본 실행에 포함하고, skip.slow.test=true일 때 제외합니다.

실행 정책

기본 실행 · skip.slow.test=true 제외

정합성과 회원 식별 처리

- 만료 대상 수집, 날짜 계산, 기업 회원 필터, 만료 알림 레코드 일괄 저장 순서를 검증했습니다.
- point_core 6개와 point_service 4개 테이블의 아카이브 흐름과 테스트를 추가했습니다.
- 같은 락 키와 서로 다른 거래 참조값, 같은 예산을 둘러싼 동시 요청을 각각 검증했습니다.
- 유료·무료 원장 혼합 사용 케이스를 별도로 검증했습니다.
- 회원-워크스페이스 매핑, master/member 분리, 회원 탈퇴 이벤트, globalUserRef 중복 처리를 보강했습니다.

03

FEED & SEARCH · SERVING / RETRIEVAL

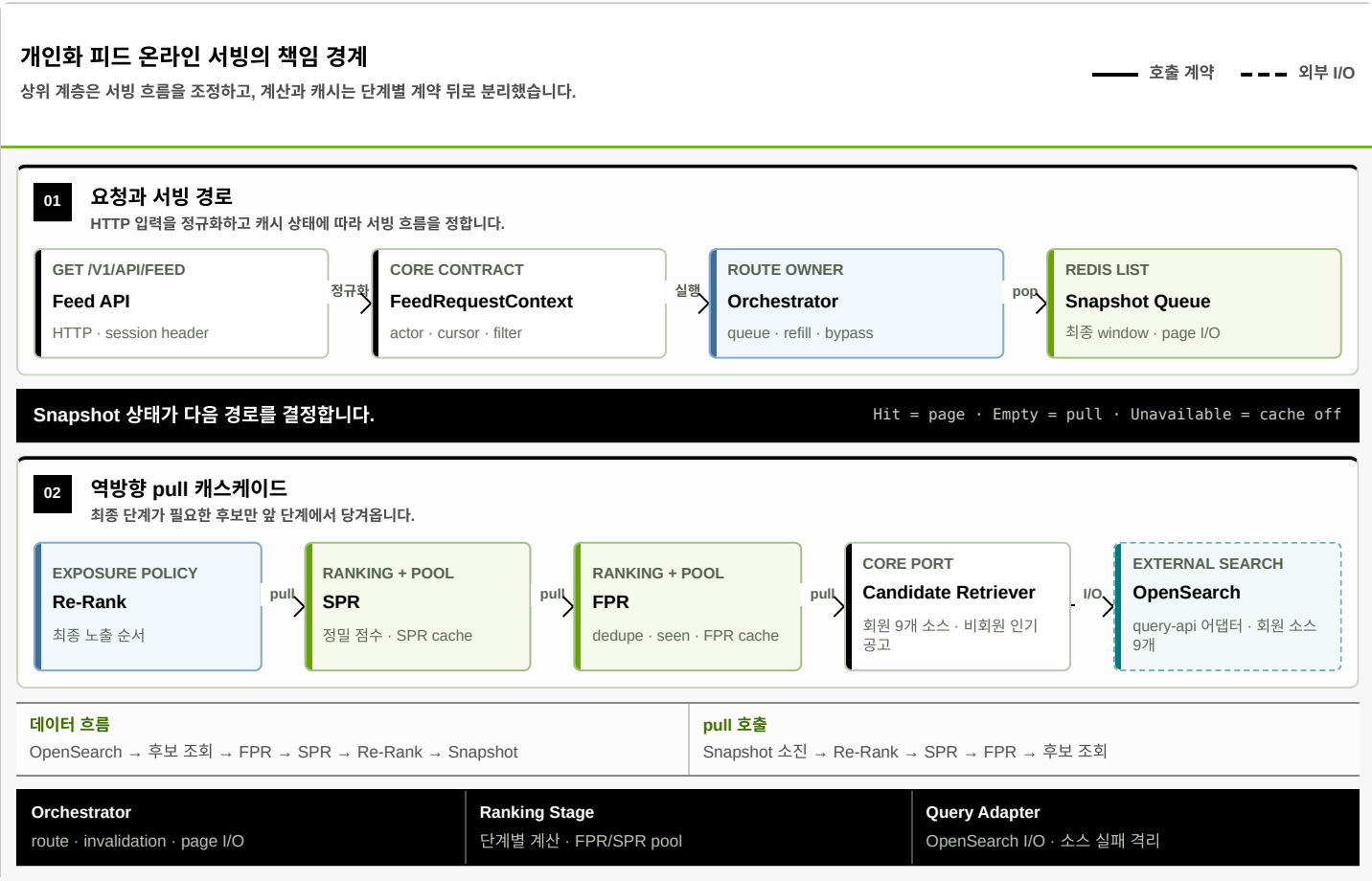
후보 수보다, 검색·랭킹·서빙의 경계를 먼저 설계했습니다.

개입화 채용 피드의 온라인 경로를 후보 수집, FPR, SPR, Re-Rank, Snapshot으로 나눴습니다. 세 랭킹 단계는 같은 pull 계약으로 있고, OpenSearch 조회와 Redis 캐시는 포트 뒤로 분리했습니다.

9개 검색 소스 후보 조회 포트	FPR · SPR · Re-Rank 공통 pull 계약	Snapshot · FPR · SPR 3종 캐시 포트
-----------------------------	--	---

전체 흐름 Candidate Retrieval → FPR → SPR → Re-Rank → Snapshot	공통 계약 후보 수집 포트 · 역방향 pull 계약 · 캐시 포트 3개	검색 경계 검색 소스 9개 · 소스별 정책· 타임아웃·실패 격리	모듈 경계 query-api는 OpenSearch 연동 전용 · 순환 의존 금지
---	---	---	--

상황 후보 소스와 랭킹 정책, 캐시 수명은 서로 다른 이유로 바뀝니다. 한 오케스트레이터가 검색 쿼리와 점수 계산, Redis 처리까지 모두 알면 작은 정책 변경도 전체 피드 경로를 건드립니다.	판단 오케스트레이터는 서빙 흐름을 조정하고, 계산과 캐시는 단계별 계약 뒤로 분리했습니다. Snapshot이 비면 Re-Rank가 SPR → FPR → 후보 수집 순서로 필요한 후보를 당겨옵니다. Redis 장애 시에는 같은 랭킹 흐름을 캐시 없이 실행합니다.
---	---



바뀌는 이유가 다른 코드를 서로 떼어냈습니다.

OpenSearch 필터·정렬 규칙과 호출은 query-api에, FPR·SPR·Re-Rank 계산은 각 랭킹 단계에, Snapshot·FPR·SPR 캐시는 포트 뒤에 뒀습니다. 오케스트레이터는 캐시 상태에 따라 서빙 흐름과 페이지 입출력을 조정합니다.

- 랭킹·캐시·검색 정책 상세
- 후보 수집은 별도 조회 포트로 두고, FPR·SPR·Re-Rank는 공통 pull 계약으로 연결했습니다.
 - Snapshot·FPR·SPR 캐시 포트가 Hit·Empty·Unavailable 상태를 같은 방식으로 표현합니다.

- OpenSearch 검색 소스 9개의 필터·정렬 규칙과 사용할 벡터를 분리했습니다. 한 소스의 타임아웃이나 실패는 나머지 후보 조회에 영향을 주지 않습니다.
- Redis 장애에는 별도 랭킹을 만들지 않고 같은 흐름을 캐시 없이 실행합니다.
- query-api는 OpenSearch 연동만 말도록 격리하고, 모듈 방향과 순환 금지는 ArchUnit으로 검증합니다.
- @SearchDocument 기반 프로세서로 검색 문서 38개 필드 경로를 생성하고, 직렬화 결과와 상수를 계약 테스트로 맞췄습니다.
- 좌표와 4축 필터의 지문이 바뀌면 Snapshot-FPR-SPR만 무효화하고 세션과 global seen은 유지합니다.
- Snapshot, FPR pool, SPR pool, user vector, global seen은 서로 다른 Redis key와 TTL로 관리했습니다.

- seen은 노출한 페이지마다 기록하고 FPR에서 감점 신호로 사용했습니다.
- 위치 기준 거리 가중치와 10km 반경, 좌표가 없는 공고의 보존·제외 정책을 반영했습니다.
- 메인 4축 필터를 색인과 질의 조립에 반영하고 회원·비회원에서 같은 의미로 적용했습니다.
- 유료 후보는 별도 영역으로 떼지 않고 FPR 점수 가산에 포함했습니다.
- 재게시된 같은 공고는 FPR에서 contentHash로 중복을 제거했습니다.
- 검색 인덱스 전환, read/write 계정 경계, strict mapping 이슈를 함께 다뤘습니다.

04

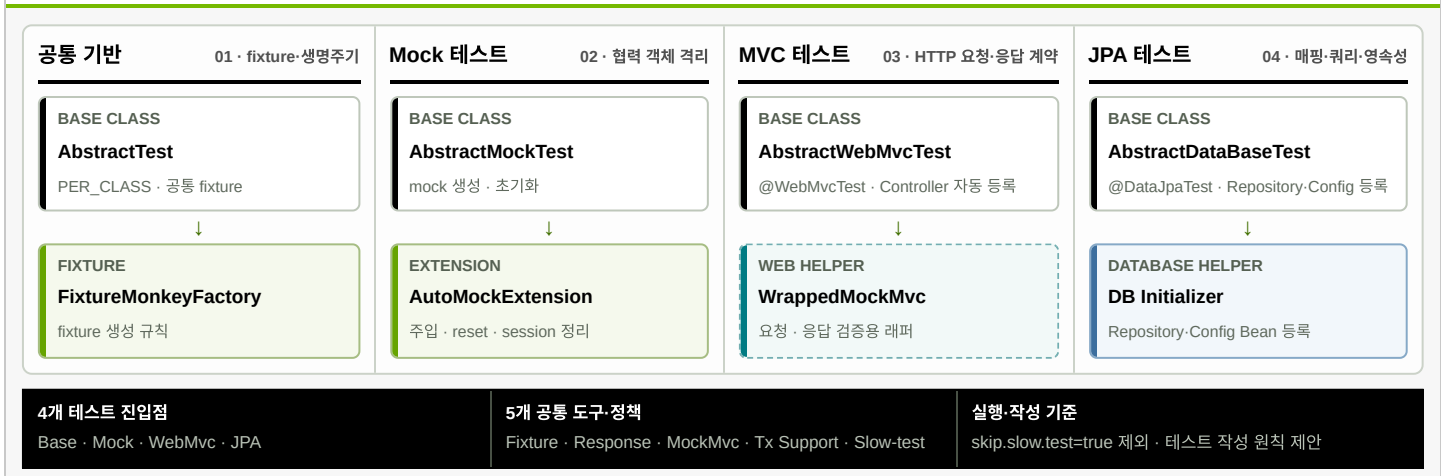
ENGINEERING PLATFORM · TEST SUPPORT

테스트마다 반복하던 준비 코드를 test-support로 모았습니다.

fixture와 mock 준비 방식이 프로젝트마다 달라 테스트 설정부터 다시 읽어야 했습니다. 네 가지 진입점과 다섯 가지 공통 도구·정책을 만들고, 테스트 선택 기준과 CI 실행 범위를 문서화했습니다.

테스트 목적에 맞춰 공통 준비를 나눈 구조

진입점, 생명주기, 컨텍스트 범위, 실행 정책을 한 기준으로 묶었습니다.



코드와 사용 기준

공통 도우미와 함께 테스트 진입점, mock·컨텍스트 생명주기, 별도 트랜잭션, 느린 테스트의 실행 기준을 문서화했습니다.

제공 기능과 CI 처리

- 기본·mock·WebMvc·JPA 네 진입점에 FixtureMonkey와 mock 생명주기를 연결했습니다.
- Controller·Repository 자동 등록, 응답 검증, REQUIRES_NEW 트랜잭션 도우미를 공통화했습니다.
- 테스트 작성 원칙을 제안하고 느린 동시성 테스트를 skip.slow.test 설정으로 분리했습니다.

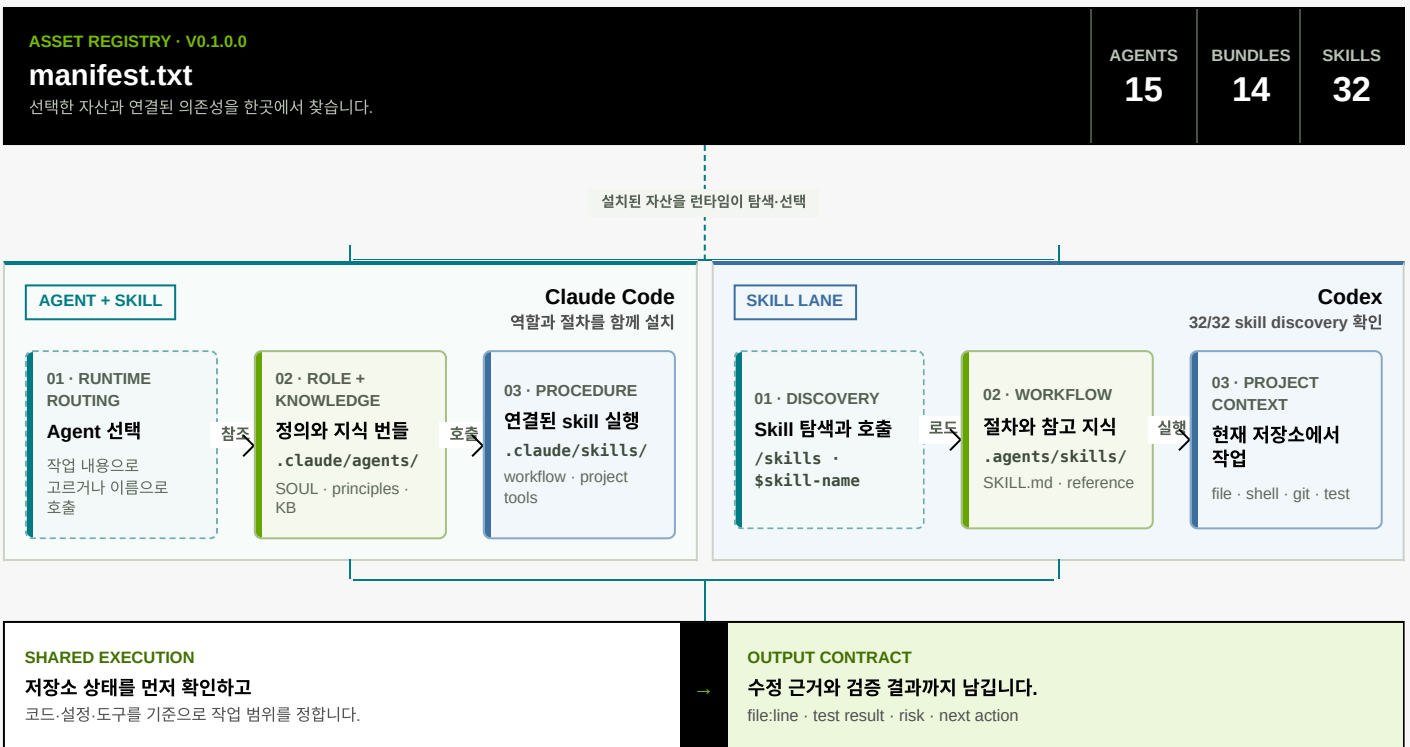
에이전트·스킬을 설치 가능한 자산으로 묶고, 구조와 행동 계약을 나눠 검증했습니다.

코드 탐색·설계 검토·테스트 절차를 15개 에이전트와 32개 스킬로 나눴습니다. manifest로 의존성을 설치하고, 구조 케이스 7개·행동 케이스 10개·설치 회귀 52개를 검증했습니다.

Agent가 역할을 정하고, skill이 절차를 제공하며, KB가 판단 근거를 보탭니다.

Agent Lab이 직접 작업을 실행하지는 않습니다. 자산과 의존성을 묶어 각 런타임에 설치합니다.

—— 실행 순서 - - - 런타임 경계



01 자산 선택 agent · skill · all-*	02 연결 항목 확인 연결 skill을 재귀 탐색	03 런타임 경로에 설치 dry-run · apply · verify	04 회귀 검사 설치 52/52 · Codex 32/32
--------------------------------------	-----------------------------------	--	---------------------------------------

OpenCode .opencode 경로 배치는 검사했습니다. agent 포맷 변환과 실제 선택·실행은 아직 검증하지 않았습니다.

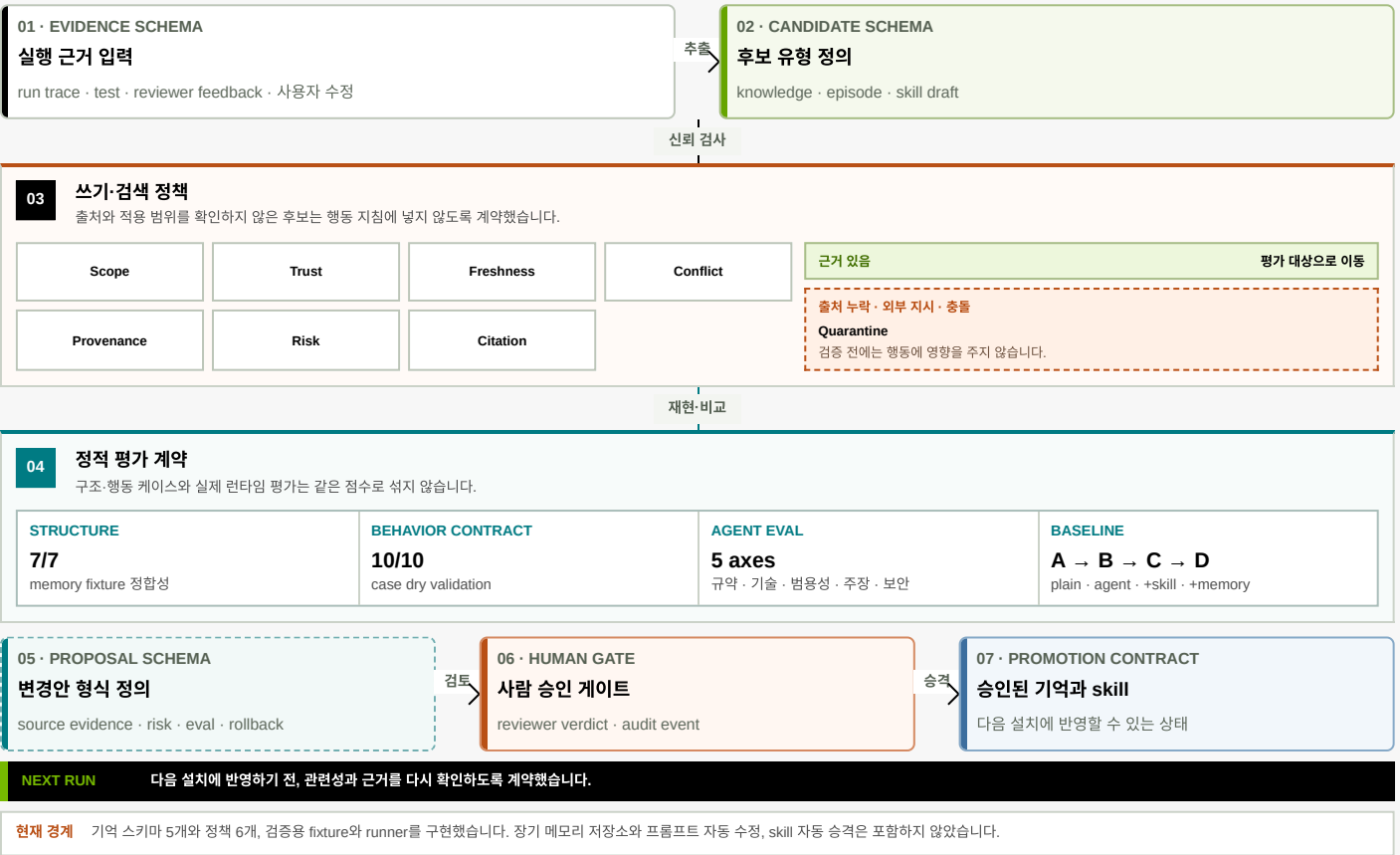
현재 설치 범위

Claude Code에는 에이전트와 스킬을, Codex에는 스킬 32개를 설치할 수 있습니다. 메모리와 자가 수정은 스키마·정책·평가 계약으로 경계를 정의한 단계이며, 실제 반영 전에 사람 승인을 요구하는 계약을 정의했습니다.

기억 후보를 검토·승인하기 위한 계약

후보가 실제 동작에 영향을 주기 전에 확인할 스키마·정책·평가 경계를 정의했습니다.

— 승격 후보 — 격리



도구 범위와 안전 경계

- manifest에 15개 agent, 14개 지식 번들, 32개 skill과 연결 의존성을 등록했습니다.
- 설치 회귀 52개를 통과했고, Codex CLI에서 32개 skill을 모두 탐색했습니다.
- 구조 fixture 7개와 행동 케이스 10개의 입력·임계값·안전 계약을 정적 검증했습니다.
- 행동 평가에서는 결과와 실행 지표를 따로 받아 결과·과정·안전·비용을 비교합니다. 실제 에이전트를 실행하려면 런타임별 adapter가 더 필요합니다.
- 메모리 5개 schema와 6개 policy로 기억을 쓰고 읽는 조건, 격리·승격·교체, 자기 수정의 경계를 먼저 정했습니다.
- NL-SQL MCP에 multi-dialect와 읽기 전용 경계를 두고 위험한 쿼리 실행을 막았습니다.
- code cartographer와 diagram extract skill로 코드 이해 결과를 다이어그램으로 공유하게 했습니다.

테스트, 인증, 검색 경험이 지금의 설계 기준이 되었습니다.

실제 구현을 검증하는 테스트, 인증·인가 경계, OpenSearch 동적 쿼리를 직접 다뤘습니다. 이 경험은 이벤트 신뢰성, 배치 재시작, 피드 캐시와 검색 정책을 설계할 때의 기준이 되었습니다.

토스랩 · JANDI

2024.06 — 2025.04

커버리지 70%인데도 배포가 불안한 이유를 테스트 기준과 CI 실행 방식에서 찾았습니다. Drive-Service를 Spring Boot 2.3.8 → 3.3.3으로 올리며 테스트를 0 → 160개 이상으로 늘렸고, Mock 테스트가 놓친 JPA 변경 감지 오류는 실제 구현 테스트로 재현해 약 200만 건의 데이터를 보정했습니다.

오케스트로 · IDP / Vista

2023.09 — 2024.05

신용정보원 IDP에서 Spring Cloud API Gateway, Spring Security·JWT 인증/인가, SonarQube·Jenkins 연동 API를 개발했습니다. 하나금융그룹 Vista에서는 VM 메트릭 조회 API와 OpenSearch 동적 쿼리 분기를 구현했습니다.

즐거운 · Backend

2022.11 — 2023.02

자사 서비스의 백엔드 기능을 개발하고 운영 중 발생한 문제를 해결했습니다.

구현 과정과 판단 근거를 글·발표·오픈소스로 공개했습니다.

기술 글 156편, YouthCon 발표, Spring OSS 개선 3건으로 테스트와 데이터 복구 경험을 공유했습니다.

YouthCon 2025 연사 '우리팀을 위한 Spring Test 만들기'에서 테스트 코드를 줄이는 기준과 팀의 그라운드 룰을 만드는 과정을 발표했습니다.	기술 글 156편 · 2026.07 기준 Java, JPA, 테스트, SQL 튜닝을 중심으로 시행착오를 기록했습니다. 글또 10기에서는 백엔드·인프라 빌리지 준비위원과 연사로 참여했습니다.
Spring OSS · 개선 제안 3건 반영 Kafka, Boot, Data JPA에 제안한 개선 3건이 반영됐습니다. 불변 컬렉션 처리, 하드코딩 제거, 자원 정리 로직을 개선했습니다.	OpenSearch 한글 분석 플러그인 JavaCafe의 Elasticsearch 플러그인을 바탕으로 한·영 자판 변환, 초성·자모 검색을 OpenSearch 2.19와 Kotlin으로 옮기고 16개 예시로 확인했습니다.
화면에 없던 7만+ 건의 오류 응답은 정상이었지만 S3에서는 삭제되고 DB에는 남은 데이터가 6년 동안 7만 건 넘게 쌓여 있었습니다. 이를 발견하고 원인을 좁혀 간 과정을 공개했습니다.	테스트로 찾은 약 200만 건 Mock 테스트가 놓친 JPA 변경 감지 오류를 실제 구현 테스트로 재현했습니다. 보정 스크립트를 작성해 약 200만 건의 데이터를 바로잡았습니다.